

7.1

为什么要用 FreeRTOS (7大痛点)

① 实时性

{ 硬实时
软实时

RTOS → 调度算法 → 任务按时完成

② 任务调度 与 多任务处理

{ 抢占式调度
优先级调度

{ 并发
切换
同步

- 简单任务 → 用户自己写调度逻辑
- 复杂任务 → RTOS 替你调度

③ 时间管理 与 延迟控制

{ 定时器
延时函数
时间片轮转

④ 资源管理 与 内存保护

{ 内存池

⇒ 避免 { 竞态
内存碎片

堆栈管理

堆栈溢出

⑤ 任务间通信与同步

裸机 { 共享内存
信号量

RTOS { 消息队列
信号量与互斥锁
事件标志

⑥ 节能与低功耗

{ 空闲任务管理
动态优先级

⑦ 调试与维护

{ 任务监视
调试工具

6.2

FreeRTOS 两种调度方式

① 时间片轮转

相同优先级任务按时间片切换

实现：链表中有多个任务，从表头取，放表尾

② 优先级抢占

高优先任务一旦就绪就抢占低优任务

实现：当 SysTick 中断 / 任务状态变化 (阻塞/解阻塞)

比较当前和新就绪任务优先级。

核心数据结构

任务控制块 TCB

- 保存任务运行所需信息
 - 栈指针 (上下文)
 - 任务状态
 - 优先级
 - 链表节点指针

+

就绪任务列表 Ready List

- 每个优先级对应一个链表

6.3

FreeRTOS 任务状态

① Ready

任务已具备运行条件(不阻塞、不挂起)
但 CPU 被更高优先任务占用

② Running

单核同一时间只能有一个任务 running

③ Blocked

任务等待事件发生或延时到期

④ Suspended (挂起态)

被挂起的任务不会被调度

⑤ Deleted

任务等待空闲任务释放其 TCB 和栈
(Idle Task) 任务控制块

7.5

FreeRTOS 如何保证实时性

① 抢占式调度 确保 高优先级任务执行

支持NVIC中断控制

- ② SysTick、时间片轮转 确保 → 任务切换时间固定且可预测
- ③ 带优先级继承的互斥锁 避免 → 优先级反转

7.6

FreeRTOS

VS

Linux

定位

实时OS

通用OS

支持多用户、多任务、网络、多种硬件架构

用途

MCU, 高实时性
资源受限设备

大型系统、PC、服务器

内核

极简内核
只有任务调度和同步
无虚拟内存

宏内核

支持虚拟内存

任务调度

基于优先级的抢占式调度
实时性、确定性强

CFS、

复杂调度策略

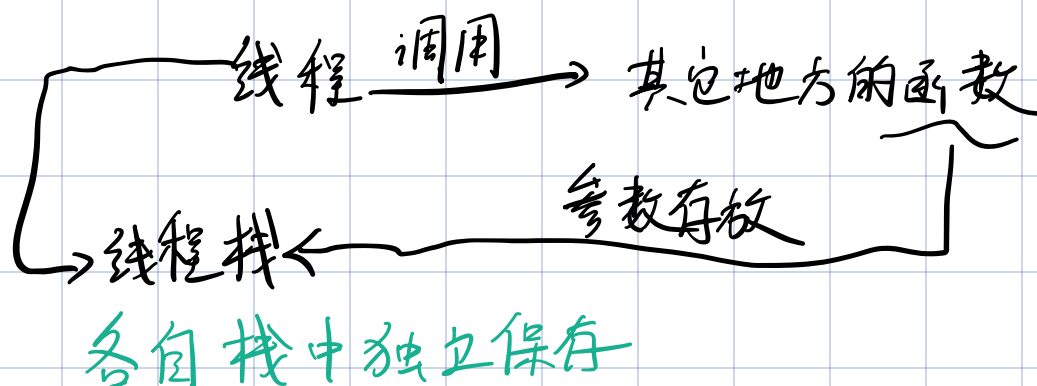
实时性

强

不是硬实时系统
存在调度延迟

7.7

多线程时



7.8

线程栈 具有 线程安全性

- 线程栈每个线程私有
- 其它线程不可访问/修改

{ 内存独立
权限控制
区域调用

何时不安全

- 函数调用传递的指针指向其它栈空间
- 栈溢出
- 使用 静态/全局变量 时
多线程共享

7.9

FreeRTOS 线程切换

保存当前任务上下文



更新当前任务状态

阻塞

就绪



选择新任务

→ 根据优先级



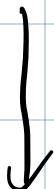
恢复新任务上下文



切换堆栈指针

→ 到新任务栈顶

此时实现了“任务切换”



跳转新任务执行

→ 根据 Program Counter

PC

7.10

动态内存管理算法

① First Fit (首次适配)

从头扫描链表, 分配第一个足够大的空闲块

快速, 高效

易产生小内存碎片

② Next Fit 循环首次适配

从上次分配处扫描链表, 分配第一个足够大的空闲块

省时间

易产生小内存碎片

③ Best Fit 最佳适配

扫描整个空闲链表, 找最小的足够大空闲块

减少内存浪费, 利用率高

时间长, 碎片集中

④ Worst Fit 最差适配

分配最大空闲块

理论上减少碎片

效果不佳, 易产生大碎片

(5)

Buddy system 伙伴系统

将内存划分为 2^k 大小的块，分配时不断拆分/合并相邻的“伙伴”块

速度快，减少碎片

内存块受限于 2^k ，可能有浪费

7.11

内存碎片解决

① 内存池

② 内存紧缩

Buddy system

Best Fit

③ 分配算法

④ 重启/内存回收

7.12

内存问题

内存碎片

内存泄露：配对申请释放 / 智能指针 / Valgrind / 工具定位

内存竞争：互斥锁，原子操作，信号量同步

越界访问：边界检查

重复释放：释放后置空指针 / 智能指针

分配失败

大大

7.13

FreeRTOS 内存管理方法

管理文件	核心分配原理	支持内存释放	内存碎片	空闲块合并	适用场景	优点
heap_1.c	静态数组分配，单向递增分配内存	❌ 不支持	❌ 无碎片	-	系统启动后不再删除任务、队列、信号量等；小型固定场景嵌入式系统	实现极简、无碎片、运行效率极高、无内存泄漏风险
heap_2.c	链表管理空闲块，采用首次适配算法	✅ 支持	✅ 严重碎片	❌ 不合并相邻空闲块	频繁创建/删除固定大小内存块；任务/队列大小固定的场景	支持释放，实现比 heap_4 简单
heap_3.c	直接封装编译器标准库 malloc() / free()	✅ 支持	取决于标准库	取决于标准库	需要复用已有 C 库内存管理；对堆大小无严格控制的场景	无需手动配置堆大小，兼容标准 C 库
heap_4.c	链表 + 最优适配算法，释放后合并相邻空闲块	✅ 支持	✅ 碎片极少	✅ 自动合并	绝大多数通用场景；频繁创建/删除不同大小内存块	碎片控制优秀，稳定性高，FreeRTOS 最常用方案
heap_5.c	在 heap_4 基础上，支持多块非连续内存区域分配	✅ 支持	✅ 碎片极少	✅ 自动合并	芯片内存分散（内部 RAM + 外部 RAM）；非连续物理内存场景	支持跨区域分配内存，内存利用率最大化

简要选型建议

1. 极简固定系统 → 选 heap_1
2. 只释放固定大小内存 → 选 heap_2
3. 通用嵌入式项目 → 优先选 heap_4
4. 多块非连续物理内存 → 选 heap_5
5. 依赖标准库内存管理 → 选 heap_3

同步/互斥机制对比

Linux

④

互斥锁 信号量 原子操作 临界区关中断

FreeRTOS

在 RTOS 中
自带优先级
继承

内存屏障

④

条件变量 自旋锁 读写锁

队列 任务通知 事件组

③

7.15 线程/任务间正确性？

正确性 避免 → { 数据竞争
资源冲突
不一致性

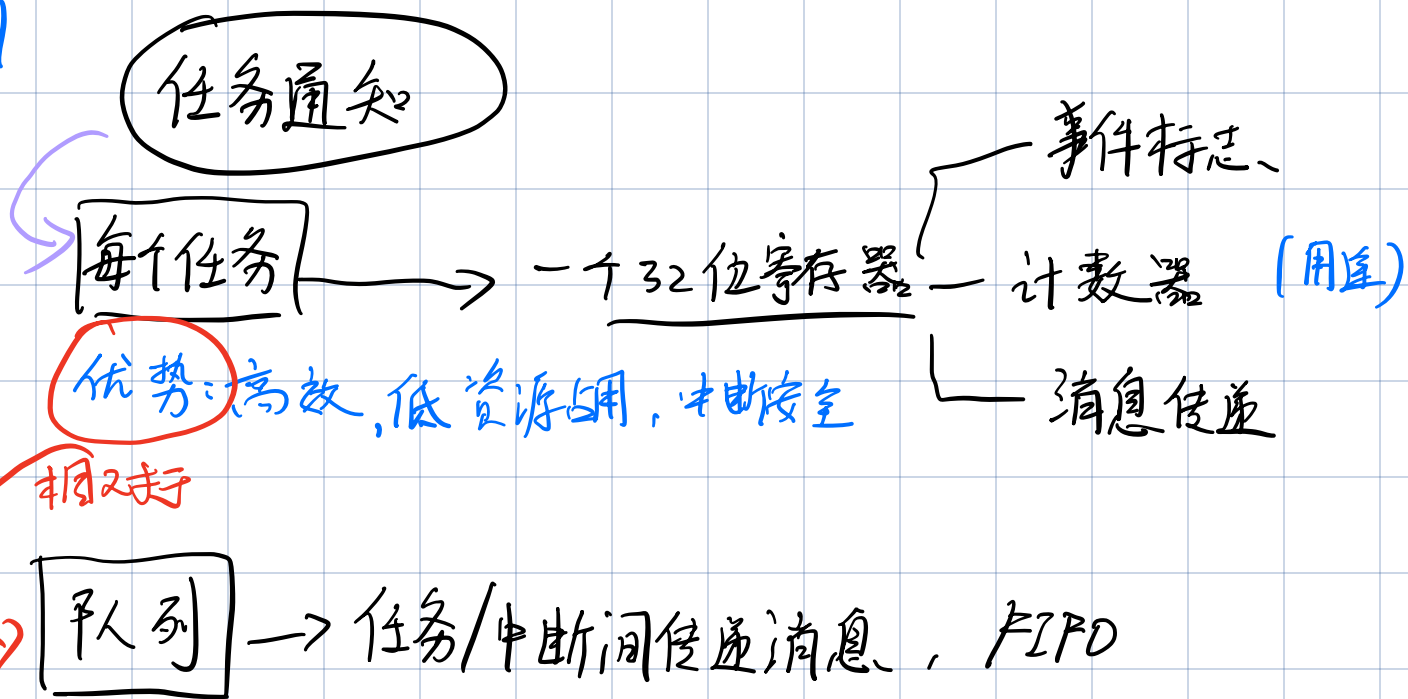
方法 { ① 互斥锁
② 信号量
③ 条件变量
④ 原子操作
⑤ 内存屏障
⑥ 减少共享变量

7.16

常见的线程同步机制

- ① 互斥锁
- ② 信号量
- ③ 条件变量
- ④ 队列
- ⑤ 原子操作

7.17



信号量 → 任务同步

互斥 → 控制资源

7.18

RTOS中哪些事件触发任务通知

- ① 任务进入 ready 态
- ② 任务优先级变化
- ③ 任务调用阻塞接口返回
- ④ 任务显式让出 CPU
- ⑤ SysTick 中断

7.19

FreeRTOS 如何避免优先级反转

- 优先级继承
- 不长时间持有资源
- 合理设计优先级

7.2.1

互斥锁 (区别)

信号量

用途

保护资源
防止并发访问

任务同步
事件通知
计数控制

拥有者概念

有

无

优先级继承

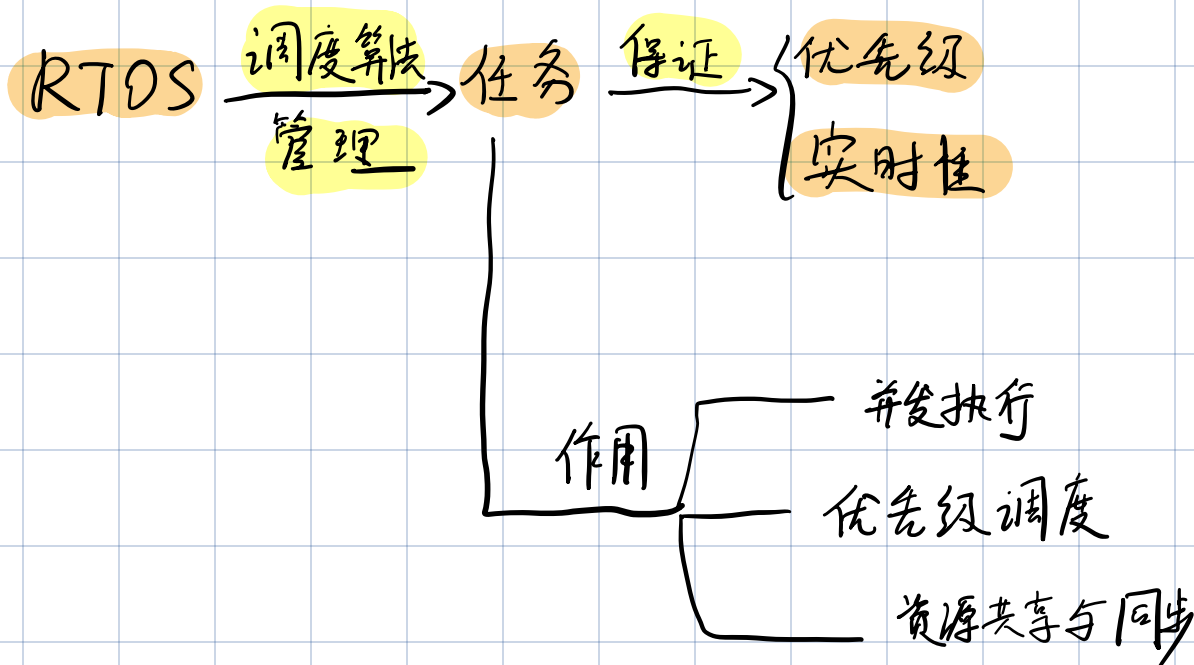
有

无

(任务等待外部事件, 收到信号量后继续执行)

7.2.2

① 任务: RTOS 中最基本的执行单元, 可以是一个函数或一组指令



作用

- 异步时间处理
- 优先级处理
- 触发调度

任务与中断:

- 中断能触发某个任务, 如 ISR 任务
- 任务被中断打断后挂起, 中断处理后恢复

作用

- 定时执行任务
- 防止任务超时
- 时间片轮转

任务与定时器:

- 定时器事件会触发任务
- 用定时器设置任务延时和防止任务超时

7.25

FreeRTOS 临界区

定义

一段必须原子操作的代码区域

作用

资源保护, 防止竞态/数据不一致

实现

taskENTER_CRITICAL() 关闭中断

↓
临界区操作

↓

7.26 哪些操作需进入临界区

- ① 访问全局/共享变量
- ② 修改内核数据结构 (优先级链表)
- ③ 队列/信号量操作的关键保护
- ④ 调度器更新任务状态
- ⑤ 中断服务函数

7.27 临界区和互斥锁的区别

临界区：通过关闭中断/禁止调度实现，执行期间不会任务切换，适合短代码。

互斥锁：基于信号量实现互斥机制，支持优先级继承，锁被占用会进入阻塞状态，适合保护复杂资源。